



本小节内容

选择排序原理解析

选择排序代码实战

选择排序原理解析

选择排序分为

1、简单选择排序

2、堆排序 ——重要

简单选择排序原理：假设排序表为 $L[1 \cdots n]$ ，第 i 趟排序即从 $L[i \cdots n]$ 中选择关键字最小的元素与 $L[i]$ 交换，每一趟排序可以确定一个元素的最终位置，这样经过 $n - 1$ 趟排序就可使得整个排序表有序。

首先假定第零个元素是最小的，把下标 0 赋值给 $\min$ ($\min$ 记录最小的元素的下标)，内层比较时，从 1 号元素一直比较到 9 号元素，谁更小，就把它的下标赋给 $\min$ ，一轮比较结束后，将 $\min$ 对应位置的元素与元素 i 交换，如下表所示。第一轮确认 2 最小，将 2 与数组开头的元素 3 交换。第二轮我们最初认为 87 最小，经过一轮比较，发现 3 最小，这时将 87 与 3 交换。持续进行，最终使数组有序。

表 选择排序过程

最初	3 87 2 93 78 56 61 38 12 40
第 1 轮比较后标记	3 87 2 93 78 56 61 38 12 40
第 1 次交换	2 87 3 93 78 56 61 38 12 40
第 2 轮比较后标记	2 87 3 93 78 56 61 38 12 40
第 2 次交换	2 3 87 93 78 56 61 38 12 40
第 3 轮比较后标记	2 3 87 93 78 56 61 38 12 40
第 3 次交换	2 3 12 93 78 56 61 38 87 40
.....	

接下来我们借助动画网站来理解一下

<https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

动画使用方法是，先点 play，然后及时点击 pause，自己通过 Step Forward 来查看

选择排序代码实战

上面已经解析了选择排序的原理，接下来我们直接进行代码实战，代码实战步骤是随机 10 个元素，然后打印，接着进行选择排序，然后打印排序后结果，代码如下：

```
#include <stdio.h>
```



```
#include <stdlib.h>
#include <time.h>
#include <string.h>
typedef int ElemType;
typedef struct{
    ElemType *elem;
    int TableLen;
}SSTable;

void ST_Init(SSTable &ST,int len)//申请空间，并进行随机数生成
{
    ST.TableLen=len;
    ST.elem=(ElemType *)malloc(sizeof(ElemType)*ST.TableLen);
    int i;
    srand(time(NULL));
    for(i=0;i<ST.TableLen;i++)
    {
        ST.elem[i]=rand()%100;
    }
}
void ST_print(SSTable ST)
{
    for(int i=0;i<ST.TableLen;i++)
    {
        printf("%3d",ST.elem[i]);
    }
    printf("\n");
}
void swap(ElemType &a,ElemType &b)
{
    ElemType tmp;
    tmp=a;
    a=b;
    b=tmp;
}
void SelectSort(ElemType A[],int n)
{
    int i,j,min;//min 记录最小的元素的下标
    for(i=0;i<n-1;i++)//最多可以为 8
    {
        min=i;//认为第 i 个位置最小
        for(j=i+1;j<n;j++)//j 最多可以为 9
        {
            if(A[j]<A[min])

```



```
        min=j;
    }
    if(min!=i)
    {
        //遍历完毕找到最小值的位置后，与 A[i]交换，这样最小值被放到了最前面
        swap(A[i],A[min]);
    }
}
}
```

//《王道 C 督学营》课程

//选择排序

```
int main()
{
    SSTable ST;
    ST_Init(ST,10);//初始化
    //下面两行为了降低调试难度，每次数组数据固定而设计的
    //ElemType A[10] = { 64, 94, 95, 79, 69, 84, 18, 22, 12, 99 };
    //memcpy(ST.elem,A,sizeof(A));
    ST_print(ST);
    SelectSort(ST.elem,10);
    ST_print(ST);
    return 0;
}
```

选择排序虽然减少了交换次数，但是循环比较的次数依然和冒泡排序的数量是一样的，都是从 1 加到 $N-1$ ，总运行次数为 $N(N-1)/2$ 。我们忽略循环内语句的数量，因为我们在计算时间复杂度时，主要考虑与 N 有关的循环，如果循环内交换得多，例如有 5 条语句，那么最终得到的无非是 $5n^2$ ；循环内交换得少，例如有 2 条语句，那么得到的就是 $2n^2$ ，但是时间复杂度计算是忽略首项系数的，因此最终还是 $O(n^2)$ 。因此，选择排序的时间复杂度依然为 $O(n^2)$ 。因为未使用额外的空间（额外空间必须与输入元素的个数 N 相关），所以空间复杂为 $O(1)$ 。另外考研初试问时间复杂度，直接写最终结果即可，不用分析过程，除非清晰说明需要给出计算过程，或者分析过程（但是目前一直都没有这个要求）。